INFORME DE LA GUIA PRÁCTICA

I. PORTADA

Tema:	Listas Secuenciales.
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero- A
Alumnos participantes:	Miranda Baño Rolando Sebastian Moyolema Criollo Katherine Lissette Medina Chico Darlyn Monserrath Ortiz Cunalata Alina Elizabeth ……
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Determinar el cálculo de la media de valores estructurados utilizando programación orientada a objetos y vectores.

Específicos:

- Diseñar e implementar una clase que represente a los estudiantes, utilizando programación orientada a objetos para organizar sus datos y calificaciones.
- Utilizar vectores para almacenar y gestionar la información de los estudiantes y sus notas de manera estructurada.
- Desarrollar funciones que permitan calcular correctamente la media (promedio) de las calificaciones tanto a nivel individual como del conjunto de estudiantes.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

Implemente una aplicación para la gestión de los datos de los estudiantes matriculados en un curso con cupo para hasta 20 estudiantes. Para ello, deberá:

1. Definir la clase Estudiante, con atributos cédula de identidad, nombres, apellidos, fecha de nacimiento y un vector con las notas (máximo de 7 notas, asumiendo que es la cantidad máxima de actividades en que se evaluará a un estudiante).
2. Implementar adecuadamente el constructor de la clase Estudiante y un método buscar(), en la clase que considere oportuno, para que busque por cédula a un estudiante dado.
3. Implemente el siguiente menú de opciones en la aplicación Java por consola: === GESTOR DE PERSONAS ===

1 Estudiantes.
2 Registro de calificaciones.
3 Determinar el promedio de notas de un estudiante.
4 Determinar el promedio de notas del curso.

Teclee su opción (1-4) El comportamiento del programa cuando el usuario interactúe con el menú de opciones deberá ser como se describe a continuación.

En la opción 1, al usuario se le mostrará un listado con un autonumérico, para identificar cada registro de cada estudiante, y los datos de cada estudiante previamente registrado. A partir de ello, el usuario podrá acceder a un submenú que le permita ingresar, modificar y eliminar los datos de un estudiante. Cada vez que un usuario inserte/modifique/elimine un registro, si es que tiene sentido, se le preguntará si desea realizar la misma acción una vez más. Si la cantidad máxima de estudiantes ya fue registrada, no se permitirá insertar datos de un nuevo estudiante. Si no hay estudiantes registrados, no se le permitirá intentar eliminar el registro de un estudiante. En todo caso, para las opciones de modificar y eliminar, el usuario comenzará indicando el valor de autonumérico que identifique al registro de estudiante que se quiere modificar/eliminar.

En la opción 2, el usuario podrá ingresar los datos de calificaciones de un estudiante. Comenzará introduciendo el número de cédula del estudiante en cuestión. Si el número de cédula pertenece a uno de los estudiantes registrados, se mostrarán sus datos (nombres y apellidos y edad), y se permitirá ingresar tantos valores de calificaciones como el usuario considere. Se le mostrará un listado de calificaciones previamente registradas. Se permitirá modificar o eliminar cada una de estas, o insertar nuevas calificaciones. Si se llega a ingresar la cantidad máxima de calificaciones posible, el sistema notificará que se han ingresado todas las calificaciones posibles y se da por terminado el proceso de entrada de calificaciones. Si el número de cédula ingresado inicialmente no fuera el de alguno de los estudiantes registrados, se le notificará oportunamente al usuario y se le dará la opción de ingresar otro número de cédula o regresar al menú principal.

En la opción 3, el usuario comenzará introduciendo el número de cédula del estudiante del que se quiere calcular el promedio de calificaciones. Si el número de cédula pertenece a uno de los estudiantes ya registrados, se mostrarán sus datos (nombres y apellidos, edad y promedio de calificaciones). Caso contrario, se emitirá un mensaje de error indicando que no se encontró un estudiante con el número de cédula indicado.

En la opción 4, se mostrará un mensaje indicando el valor del promedio general de calificaciones. Si ningún estudiante tiene calificaciones registradas, de igual manera, se indicará que No se han registrado calificaciones de estudiantes. Nota: el conjunto de estudiantes, y el conjunto de notas por estudiante, se almacenará en vectores

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

· Inteligencia artificial, TAC

PC de escritorio o portátil con JDK instalado e IDE de su elección

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☐Plataformas educativas
☐Simuladores y laboratorios virtuales
☐Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☐Inteligencia Artificial
Otros (Especifique): _____

2.6 Actividades por desarrollar

Implementación de aplicación para la gestión de los datos de los estudiantes matriculados en un curso con cupo para hasta 20 estudiantes.

2.7 Resultados obtenidos

Se desarrollan habilidades en la solución a un problema de baja complejidad utilizando programación orientada a objetos y vectores.

1. Introducción

En el presente trabajo se desarrolló una aplicación en **C++** que permite gestionar los datos de estudiantes y sus calificaciones mediante el uso de programación orientada a objetos y estructuras de datos tipo vector.

El sistema facilita el registro, modificación y eliminación de estudiantes, así como el manejo de sus calificaciones y el cálculo de promedios, permitiendo una gestión eficiente de la información.

3. Marco Teórico *(Alina)*

La programación orientada a objetos (POO) es un paradigma de programación que permite organizar el código en estructuras llamadas clases, las cuales agrupan datos (atributos) y comportamientos (métodos). Este enfoque facilita la reutilización del código, su mantenimiento y escalabilidad.

En este proyecto se utiliza la clase Estudiante, que representa una entidad real, permitiendo modelar sus características como cédula, nombres, apellidos, fecha de nacimiento y calificaciones. A través de métodos, se pueden realizar operaciones como calcular el promedio de notas.

Por otro lado, los vectores (arreglos) son estructuras de datos que almacenan elementos del mismo tipo en posiciones contiguas de memoria. En este caso, se utilizan para almacenar hasta 20 estudiantes y un máximo de 7 calificaciones por cada uno. Su uso permite acceder rápidamente a los datos mediante índices.

Además, se aplican conceptos importantes como:

- Búsqueda secuencial: para localizar un estudiante por su cédula dentro del vector.
- Validación de datos: para asegurar que la información ingresada sea correcta (por ejemplo, formato de cédula o rango de notas).
- Estructuras de control: como ciclos y condicionales, que permiten manejar la lógica del programa y la interacción con el usuario mediante menús.

Este conjunto de herramientas permite desarrollar una aplicación funcional, organizada y eficiente para la gestión de datos académicos.

4. Desarrollo del Sistema *(Alina)*

El sistema fue desarrollado en el lenguaje de programación C++, utilizando el paradigma de programación orientada a objetos. Se diseñó con el objetivo de gestionar de manera eficiente la información de estudiantes y sus calificaciones dentro de un curso.

Para la implementación, se creó la clase *Estudiante*, que contiene los atributos necesarios para almacenar la información personal y académica. Además, se implementaron métodos dentro de esta clase para calcular el promedio de notas y gestionar los datos.

El programa utiliza vectores como estructura principal de almacenamiento, permitiendo manejar un número limitado de estudiantes (máximo 20) y calificaciones (máximo 7 por estudiante), tal como lo establece la guía práctica.

La interacción con el usuario se realiza mediante un menú en consola, el cual permite acceder a diferentes funcionalidades como:

- Gestión de estudiantes
- Registro de calificaciones
- Cálculo de promedios

También se implementaron diversas validaciones para evitar errores, como:

- Verificación del número de cédula
- Control del rango de notas (0 a 10)
- Límites máximos de almacenamiento

En conjunto, el sistema cumple con los requisitos planteados, ofreciendo una solución clara, estructurada y funcional al problema propuesto.

4.1 Descripción General del Sistema *(Rolando)*

El sistema es una aplicación de consola desarrollada en **C++** que permite gestionar información de estudiantes matriculados en un curso. Tiene un límite máximo de 20 estudiantes y cada uno puede registrar hasta 7 calificaciones.

El usuario interactúa con el sistema a través de un menú que facilita el acceso a las diferentes funcionalidades, como gestión de estudiantes, registro de notas y cálculo de promedios.

4.2 Estructura del Programa (se agrega que se uso en cada opcion) *(Rolando)*

La opción 1 (Gestión de Estudiantes) utiliza las funciones insertarEstudiante(), modificarEstudiante() y eliminarEstudiante(), agrupadas bajo menuEstudiantes(). Para insertar, se valida la cédula con validarCedula() verificando que tenga exactamente 10 dígitos numéricos, se comprueba que no exista duplicado mediante buscarEstudiantePorCedula(), y se valida la fecha de nacimiento con validarFecha() en formato DD/MM/AAAA. Para modificar o eliminar, el usuario selecciona el estudiante por su número en la lista y se confirma la acción antes de aplicarla. En todo momento se controla que no se supere el límite de 20 estudiantes ni se intente eliminar cuando la lista está vacía.

La opción 2 (Registro de Calificaciones) usa la función registroCalificaciones(). El usuario ingresa una cédula que es buscada con buscarEstudiantePorCedula(); si existe, se muestran los datos del estudiante junto con su edad calculada mediante calcularEdad(). Las notas se ingresan con leerNota(), que valida que cada valor esté entre 0.00 y 10.00, y se almacenan en la lista

notas del objeto Estudiante. El proceso finaliza automáticamente al alcanzar las 7 calificaciones máximas.

La opción 3 (Promedio Individual) emplea la función promedioEstudiante(). Busca al estudiante por cédula y, si es encontrado, llama al método calcularPromedio() de la clase Estudiante, el cual recorre la lista de notas, suma los valores y los divide entre la cantidad de notas registradas, mostrando el resultado junto con los datos del estudiante.

La opción 4 (Promedio del Curso) usa la función promedioCurso(), que recorre todos los estudiantes registrados, acumula sus notas y calcula el promedio general. Si ningún estudiante tiene calificaciones registradas, muestra un mensaje indicándolo.

4.3 Explicación de las Opciones del Menú

El sistema desarrollado cuenta con un menú principal que permite al usuario interactuar con las diferentes funcionalidades de la aplicación. A continuación, se describe el funcionamiento de cada opción:

Opción 1: Estudiantes
Permite gestionar la información de los estudiantes almacenados en el sistema. Se muestra un listado con un identificador autonumérico. El usuario puede:

- Ingresar nuevos estudiantes
- Modificar datos existentes
- Eliminar registros

Se valida que no se exceda el límite de 20 estudiantes y que existan datos antes de permitir eliminaciones.

Opción 2: Registro de calificaciones
Permite gestionar las notas de un estudiante. El usuario ingresa la cédula y, si el estudiante existe:

- Se muestran sus datos
- Se pueden agregar, modificar o eliminar calificaciones

El sistema controla que no se superen las 7 notas por estudiante. Si la cédula no existe, se muestra un mensaje de error.

Opción 3: Promedio de un estudiante
Permite calcular el promedio de un estudiante específico. Se ingresa la cédula y:

- Si existe → muestra datos + promedio
- Si no existe → muestra error

Opción 4: Promedio del curso
Calcula el promedio general de todas las calificaciones registradas en el sistema.
Si no existen notas registradas, el sistema muestra un mensaje indicando esta situación.

4.4 Lógica del Programa

El sistema funciona mediante un menú interactivo en consola que permite al usuario seleccionar diferentes opciones. Se utiliza un ciclo repetitivo (como while o do-while) para mantener el programa en ejecución hasta que el usuario decida salir.

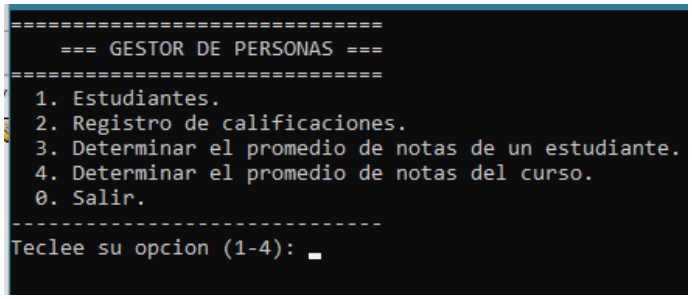
Los datos de los estudiantes se almacenan en un arreglo o vector con capacidad máxima de 20 elementos, mientras que las calificaciones se almacenan en arreglos asociados a cada estudiante con un máximo de 7 notas.

El programa implementa validaciones para evitar errores, como la verificación de existencia de estudiantes, control de límites de almacenamiento y manejo de entradas incorrectas. Además, se utiliza una función de búsqueda para localizar estudiantes por su número de cédula.

4.5 Capturas del Programa

En esta sección se incluyen las capturas de pantalla que evidencian el funcionamiento del sistema, tales como:

- **Menú principal**



Explicación:
Pantalla principal del sistema donde se presentan las cuatro opciones principales: gestión de estudiantes, registro de calificaciones, cálculo de promedio individual y promedio del curso. El usuario debe ingresar un número del 1 al 4 para acceder a la funcionalidad deseada.

- **Registro de estudiantes**

```
=== INSERTAR ESTUDIANTE ===
Cedula de identidad (10 digitos): 1751771429
Nombres: Luis Esteban
Apellidos: Sanchez Sambrano
Fecha de nacimiento (DD/MM/AAAA): 10/10/2003

[OK] Estudiante registrado exitosamente.

Desea ingresar otro estudiante? (s/n): s
```

Explicación:
Ventana para insertar un nuevo estudiante. Se solicita la cédula (10 dígitos), nombres, apellidos y fecha de nacimiento en formato DD/MM/AAAA. Una vez ingresados correctamente, el sistema confirma el registro y pregunta si desea continuar agregando más estudiantes, respetando el límite máximo de 20.

```
=== GESTION DE ESTUDIANTES ===

-------------------------------------------------------------
 No. | Cedula     | Nombres          | Apellidos
-------------------------------------------------------------
   1 | 1851003929 | Katherine Lissette | Moyolema Criollo
 2   | 1751771429 | Luis Esteban     | Sanchez Sambrano
 3   | 2767141312 | Martin Mateo     | Soto Soto
-------------------------------------------------------------

 1. Ingresar estudiante
 2. Modificar estudiante
 3. Eliminar estudiante
 4. Regresar al menu principal

Teclee su opcion (1-4):
```

Explicación:
Submenú de gestión de estudiantes que muestra un listado con todos los registros existentes, identificados por un número correlativo. Desde aquí el usuario puede seleccionar las opciones de ingresar, modificar o eliminar estudiantes. En la imagen se observan tres estudiantes registrados con sus respectivas cédulas, nombres y apellidos.

- **Registro de calificaciones**

```
=== REGISTRO DE CALIFICACIONES ===
Ingrese la cedula del estudiante: 1751771429

Estudiante encontrado:
  Nombres  : Luis Esteban Sanchez Sambrano
  Edad     : 22 anios

Presione Enter para continuar...
```

Explicación:
Al seleccionar la opción "Registro de calificaciones", el sistema solicita la cédula del estudiante. Una vez ingresada, si el estudiante existe, se muestran sus datos personales (nombres completos y edad calculada automáticamente a partir de su fecha de nacimiento). En este caso se encontró al estudiante Luis Esteban Sánchez Sambrano con 22 años.

```
=== CALIFICACIONES DE: Luis Esteban Sanchez Sambrano ===

  Calificaciones registradas:
  1  . 10.00
  2  . 7.00
  3  . 7.00
  4  . 6.00

  Opciones:
  1. Insertar calificacion
  2. Modificar calificacion
  3. Eliminar calificacion
  4. Regresar

Teclee su opcion: _
```

Explicación:
Pantalla donde se gestionan las calificaciones del estudiante seleccionado. Se muestra un listado numerado de las notas registradas (en este caso cuatro calificaciones: 10.00, 7.00, 7.00, 6.00). El usuario puede insertar nuevas notas (hasta un máximo de 7), modificar notas existentes o eliminarlas según lo necesite.

- **Cálculo de promedios**

```
=== PROMEDIO DE NOTAS DE UN ESTUDIANTE ===
Ingrese la cedula del estudiante: 1751771429

  Nombres  : Luis Esteban Sanchez Sambrano
  Edad     : 22 anios
  Calificaciones: 10.00, 7.00, 7.00, 6.00
  Promedio : 7.50

Presione Enter para continuar..._
```

Explicación:
Resultado del cálculo del promedio de un estudiante específico. Al ingresar la cédula, el sistema muestra los datos personales del estudiante (nombres y edad), el listado completo de sus calificaciones y finalmente su promedio calculado. En este caso, el estudiante Luis Esteban Sánchez Sambrano obtuvo un promedio de 7.50 sobre 10.

```
=== PROMEDIO GENERAL DE CALIFICACIONES DEL CURSO ===

  Total de calificaciones registradas : 12
  Promedio general del curso          : 7.83

Presione Enter para continuar...
```

Explicación:
Resultado del cálculo del promedio general de todo el curso. El sistema suma todas las calificaciones de todos los estudiantes registrados y calcula el promedio total. En la imagen se muestra que se registraron 12 calificaciones en total, obteniendo un promedio general de 7.83.

5. Resultados Obtenidos

El sistema desarrollado permite gestionar de manera eficiente los datos de los estudiantes y sus calificaciones. Se logró implementar correctamente la estructura de clases en C++, el uso de arreglos o vectores y la interacción mediante un menú.

El programa cumple con los requisitos planteados, permitiendo registrar estudiantes, gestionar calificaciones y calcular promedios individuales y generales.

2.8 Habilidades blandas empleadas en la práctica

☐ Liderazgo
☒ Trabajo en equipo
☐ Comunicación asertiva
☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☐ Adaptabilidad
☒ Responsabilidad

2.9 Conclusiones

La programación orientada a objetos facilita la estructuración del código de forma natural, facilitando el mantenimiento del código de una aplicación. El uso de vectores requiere conocer la dimensión de dichos vectores a priori, lo que limita su aplicación al implicar un uso potencialmente ineficiente de la memoria RAM.

En base al desarrollo del proyecto de gestión de estudiantes y calificaciones, se presentan las siguientes conclusiones:

- La Programación Orientada a Objetos (POO) facilitó la organización del código, permitiendo encapsular los atributos y métodos relacionados con el estudiante dentro de la clase Estudiante, lo que mejora la legibilidad y mantenibilidad del sistema.
- El uso de std::list como estructura de datos dinámica permitió gestionar eficientemente el almacenamiento de estudiantes y sus calificaciones, superando las limitaciones de los vectores estáticos al no requerir un tamaño predefinido.
- Se implementaron todas las funcionalidades solicitadas en la guía práctica, incluyendo la gestión completa de estudiantes (insertar, modificar, eliminar), el registro de calificaciones con un límite de 7 notas por estudiante, y el cálculo de promedios tanto individuales como del curso.
- Las validaciones implementadas garantizan la integridad de los datos, como la verificación del formato de cédula (10 dígitos numéricos), la validación de fechas de nacimiento (formato DD/MM/AAAA), y el control de límites máximos (20 estudiantes y 7 notas por estudiante).
- El sistema de menús anidados proporciona una experiencia de usuario intuitiva, permitiendo navegar fácilmente entre las diferentes funcionalidades y ofreciendo opciones de continuar o regresar según el contexto.
- El cálculo de edad a partir de la fecha de nacimiento y el promedio de calificaciones se implementan correctamente, mostrando resultados precisos con formato de dos decimales para una mejor presentación.

2.10 Recomendaciones

Nombrar clases y atributos de estas de forma tal que se facilite entender el uso que estos tienen, utilizar un estilo de código de programación limpia y aplicar los principios de la programación orientada a objetos.

Con base en la experiencia de desarrollo, se recomienda lo siguiente para futuras versiones del sistema:

- Implementar persistencia de datos, utilizando archivos de texto o una base de datos ligera (como SQLite), para que la información de estudiantes y calificaciones no se pierda al cerrar la aplicación.
- Reemplazar std::list por std::vector, según lo indicado en la guía práctica, para cumplir estrictamente con el requerimiento de utilizar vectores como estructura de almacenamiento principal.
- Mejorar las validaciones de entrada, añadiendo verificación de que los nombres y apellidos no contengan números o caracteres especiales, así como limitar la longitud máxima de estos campos.
- Agregar la funcionalidad de búsqueda avanzada, permitiendo filtrar estudiantes por nombre, apellido o rango de edad, facilitando la localización de registros cuando el número de estudiantes sea elevado.

2.11 Referencias bibliográficas

2.12 Anexos

Link del github: https://github.com/Monse1806/Guia_Ape_02.git